CITS5508 Practice Exam — Chapter 5 — ANSWER KEY

Question 1.

(a) *(6 marks)* A linear SVM finds the hyperplane that **maximises the margin** — the widest "street" between the two classes. - **Margin:** defined by the two parallel hyperplanes $w^T x + b = +1$ and $w^T x + b = -1$; the decision boundary is the centre line $w^T x + b = 0$. - **Support vectors:** the training instances lying **on the edges of the street** (and, in the soft-margin case, any margin violators). They alone **determine** the boundary. - **Relevant instances:** only the support vectors matter — adding/removing instances **off** the street does not change the boundary. - **Decision rule:** for a new instance, compute $w^T x + b$ and predict the **positive class if it is ≥ 0**, else the negative class (i.e. which side of the boundary it falls on). *(award ~1.5 marks per element.)*

(b) *(3 marks)* The street width equals $2/\|w\|$, so making the margin **wider** means making **$\|w\|$ smaller**. We minimise **$\frac{1}{2}\|w\|^2$** rather than maximising the margin because $\frac{1}{2}\|w\|^2$ is **differentiable everywhere** (its gradient is just w) and convex, whereas $\|w\|$ is not differentiable at 0 — optimisation works much better on the smooth quadratic.

(c) *(3 marks)* SVMs are **sensitive to feature scales**: a feature with a much larger range dominates the geometry, so the widest street is found mostly along that axis and the boundary is poor. After scaling (e.g. `StandardScaler`) all features contribute comparably and the margin is meaningful.

Question 2.

(a) *(5 marks)* **Hard margin:** requires **every** instance to be off the street and on the correct side. **Soft margin:** allows some **margin violations** (instances inside the street or on the wrong side) to obtain a wider, more robust boundary. Two problems with hard margin: (1) it **only works if the data is linearly separable**; (2) it is **very sensitive to outliers** (a single outlier can make separation impossible or badly distort the boundary).

(b) *(4 marks)* **C** controls the trade-off between a wide margin and few violations: - **Low C** → more regularisation → **wider** street, **more** violations, **more** support vectors (less overfitting, but too low → underfitting). - **High C** → narrower street, fewer violations → can **overfit**. - If overfitting, **decrease C**.

(c) *(3 marks)* `LinearSVC` does **not** provide probabilities (no `predict_proba`). It can give a **confidence score** via `decision_function()` — the **signed distance** to the decision boundary. (To get probabilities you'd use `SVC(probability=True)`, which fits an extra calibration model via CV and is slower.)

Question 3.

(a) *(4 marks)* The **kernel trick** uses a kernel function `K(a,b)` to compute the **dot product of the transformed feature vectors** $\varphi(a)^T\varphi(b)$ **directly from the original vectors**, **without** ever computing the (possibly huge or infinite-dimensional) transformation φ. Because SVM training/prediction depend on the data only through dot products, this yields the **same result** as explicitly mapping to a high-dimensional space, but far more efficiently and with **no combinatorial explosion** of features.

(b) *(4 marks)* Gaussian RBF kernel: **$K(a,b) = \exp(-\gamma \cdot \|a - b\|^2)$**. Increasing **γ** makes the bell narrower → each instance's influence is **more local** → the boundary becomes more **wiggly/irregular** (overfitting). γ acts like a regularisation knob: **if the model**

underfits, increase γ (and/or C); if it overfits, decrease γ.

(c) *(4 marks)* **Mercer's theorem:** if K satisfies Mercer's conditions (continuous, symmetric, etc.), then a transformation φ with `K(a,b) = φ(a)ᵀφ(b)` is **guaranteed to exist** — so you can use K validly **even without knowing φ**. For the **RBF kernel, φ maps to an infinite-dimensional space**, which is exactly why you must use the trick rather than computing φ explicitly.

Question 4.

(a) *(6 marks)* | Class | Time complexity | Kernel trick | Large / out-of-core | |—|—|—|—| | `LinearSVC` | ~O(m·n) | **No** | scales ~linearly with m; no out-of-core | | `SVC` | O(m²·n) to O(m³·n) | **Yes** | slow for large m → small/medium data only | | `SGDClassifier` | O(m·n) | No | **Yes** — out-of-core / incremental, low memory | *(2 marks per class, covering the three aspects.)*

(b) *(3 marks)* **`LinearSVC`** (or `SGDClassifier`). The data is **linearly separable** so no kernel is needed, and with **2 million instances** the kernelised `SVC` (O(m²–m³)) would be far too slow; `LinearSVC` scales ~linearly with m. (`SGDClassifier` is also fine and supports out-of-core.)

(c) *(3 marks)* **`SVC` with the Gaussian RBF kernel**. The dataset is **nonlinear**, so a kernel is needed; RBF is the usual first choice and works well; the set is **small (~1,000)** so `SVC`'s O(m²–m³) cost is acceptable.

Question 5.

(a) *(5 marks)* SVM regression **flips the objective**: instead of fitting the widest street *between* classes, it fits as many instances as possible **inside** a street of width controlled by **ε**, while limiting points that fall outside it. **ε** sets the **width of the margin (the tube)**. The model is **ε-insensitive** because adding training instances **within** the margin does **not** change the predictions (errors smaller than ε are ignored). *(Smaller ε = narrower tube = more support vectors = more regularisation.)*

(b) *(4 marks)*

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC

svm_clf = make_pipeline(
    StandardScaler(),
    SVC(kernel="rbf", C=10, gamma=0.5))
svm_clf.fit(X_train, y_train)
```

(c) *(3 marks)* Solve the **dual** when (any one): the **number of instances is smaller than the number of features**, or because you want to use a **kernel**. The dual expresses the problem purely in terms of **dot products between instances**, which is precisely what allows the **kernel trick** (replace each dot product by `K`); the primal does not.